

LESSON : Non-Equi Joins

What is a Non-Equi Join?

Class, a **Non-Equi Join** is a SQL join that connects two tables using a **comparison condition other than the equal sign (=)**.

In simple terms:

Non-Equi Join = a JOIN that uses comparison operators such as >, <, >=, <=, or != instead of =.

Let's Compare Equi Join and Non-Equi Join

Equi Join

Example:

ON Employee.DepartmentID = Department.DepartmentID

This means:

"Join the rows when the DepartmentID values are equal."

The condition is based on **exact matching**.

Non-Equi Join

Example:

ON Employee.Salary >= SalaryGrade.MinSalary

AND Employee.Salary <= SalaryGrade.MaxSalary

This means:

"Join the employee to the salary grade when the employee's salary falls within the minimum and maximum salary range."

Here, the values do **not have to be equal**.

Instead, we are checking whether one value **falls within a specific range**.

1. Comparison Based on Inequality

The first important characteristic of a Non-Equijoin is that it uses **comparison operators** instead of simply using =.

Common operators include:

- != — not equal
- > — greater than
- < — less than
- >= — greater than or equal to
- <= — less than or equal to

The important point is that we are **comparing values from different tables**.

SLIDE 19 — Comparison Operators

Operator	Meaning	Example
=	Equal	A = B
!=	Not equal	A != B
>	Greater than	A > B
<	Less than	A < B
>=	Greater than or equal to	A >= B
<=	Less than or equal to	A <= B

Remember:

A Non-Equijoin does not require the values to be exactly the same. It uses a comparison to determine whether rows should be joined.

2. Example: Employee and Salary Grade

Let's use a simple example.

Suppose we have two tables.

Employees

EmployeeID	Name	Salary
1	Ana	18,000
2	Ben	25,000
3	Carla	35,000
4	David	50,000

SalaryGrade

Grade	MinSalary	MaxSalary
C	10,000	20,000
B	20,001	30,000
A	30,001	40,000
S	40,001	60,000

Now our question is:

What salary grade belongs to each employee?

Why can't we simply use =?

We cannot use:

ON Employees.Salary = SalaryGrade.MinSalary

Why?

Because Ana's salary is **18,000**, while Grade C's minimum salary is **10,000**.

They are not equal.

What we really need to check is:

Does the employee's salary fall between the minimum and maximum salary of a grade?

That requires a **range comparison**.

3. Non-Equijoin Using >= and <=

In SQL Server, we can write:

```
SELECT
```

```
    E.EmployeeID,
```

```
    E.Name,
```

```
    E.Salary,
```

```
    S.Grade
```

```
FROM Employees AS E
```

```
INNER JOIN SalaryGrade AS S
```

```
    ON E.Salary >= S.MinSalary
```

```
    AND E.Salary <= S.MaxSalary;
```

Let's understand what SQL is doing.

For Ana

Ana's salary is **18,000**.

SQL checks:

18,000 >= 10,000 → TRUE

and:

18,000 <= 20,000 → TRUE

Both conditions are TRUE.

Therefore:

Ana belongs to Grade C.

For Ben

Ben's salary is **25,000**.

SQL checks:

25,000 >= 20,001 → TRUE

and:

25,000 <= 30,000 → TRUE

Both conditions are TRUE.

Therefore:

Ben belongs to Grade B.

4. Expected Result

The query produces:

EmployeeID	Name	Salary	Grade
1	Ana	18,000	C
2	Ben	25,000	B
3	Carla	35,000	A
4	David	50,000	S

Notice something important:

The employee's salary is **not equal** to the MinSalary or MaxSalary.

Instead, SQL checks **which range contains the salary**.

That is why this is called a **Non-Equijoin**.

5. Flexible Conditions

Another characteristic of a Non-Equijoin is that it is **flexible**.

We are not limited to >= and <=.

For example:

ON A.Salary > B.MinSalary

This means:

Join the rows when A's salary is greater than B's minimum salary.

Another example:

ON A.Salary < B.MaxSalary

This means:

Join the rows when A's salary is less than B's maximum salary.

We can also use:

ON A.Value != B.Value

This means:

Join the rows when the two values are not equal.

So, class, the comparison we use depends on the **problem we are trying to solve**.

6. Complex Relationships

Non-Equi Joins are also useful when the relationship between two tables is based on **ranges or other comparisons**.

For example, suppose we have a Product table.

Product

Product Price

Laptop 45,000

Mouse 800

Monitor 12,000

And a Discount table:

Discount

DiscountLevel MinPrice MaxPrice Discount

	l	e	t
Low	0	999	5%
Medium	1,000	9,999	10%
High	10,000	49,999	15%

We can use a Non-Equijoin:

SELECT

P.Product,

P.Price,

D.DiscountLevel,

D.Discount

FROM Product AS P

INNER JOIN Discount AS D

ON P.Price >= D.MinPrice

AND P.Price <= D.MaxPrice;

Result

Product	Price	DiscountLevel	Discount
Laptop	45,000	High	15%
Mouse	800	Low	5%
Monitor	12,000	High	15%

Here, we are **not** saying:

Price = MinPrice

Instead, we are checking:

MinPrice <= Price <= MaxPrice

In other words:

Which price range does the product belong to?

That is a Non-Equijoin.

7. SQL Server Example in Visual Studio

If you are using **SQL Server through Visual Studio**, you can first create the sample tables.

Create the Employees table

```
CREATE TABLE Employees
```

```
(  
    EmployeeID INT,  
    Name VARCHAR(50),  
    Salary DECIMAL(10,2)  
);
```

Create the SalaryGrade table

```
CREATE TABLE SalaryGrade
```

```
(  
    Grade CHAR(1),  
    MinSalary DECIMAL(10,2),  
    MaxSalary DECIMAL(10,2)  
);
```

Insert the employee data

```
INSERT INTO Employees VALUES
```

```
(1, 'Ana', 18000),  
(2, 'Ben', 25000),  
(3, 'Carla', 35000),  
(4, 'David', 50000);
```

Insert the salary grade data

```
INSERT INTO SalaryGrade VALUES
```

```
('C', 10000, 20000),
```

```
('B', 20001, 30000),
```

```
('A', 30001, 40000),
```

```
('S', 40001, 60000);
```

Execute the Non-Equijoin

```
SELECT
```

```
    E.EmployeeID,
```

```
    E.Name,
```

```
    E.Salary,
```

```
    S.Grade
```

```
FROM Employees AS E
```

```
INNER JOIN SalaryGrade AS S
```

```
    ON E.Salary >= S.MinSalary
```

```
    AND E.Salary <= S.MaxSalary;
```

Equijoin vs. Non-Equijoin

This is one of the most important comparisons to remember.

Equijoin

Uses =

Exact matching

A.ID = B.ID

Usually used for direct relationships

Example: Student and Course

Non-Equijoin

Uses operators other than =

Comparison or range matching

A.Salary >= B.MinSalary

Useful for ranges and inequalities

Example: Employee and Salary Grade

Easy way to remember

Equijoin asks:

"Are the values equal?"

A.Value = B.Value

Non-Equijoin asks:

"Does it fit the range? Is it greater? Is it smaller? Or is it not equal?"

A.Value >= B.MinValue

or

A.Value <= B.MaxValue

or

A.Value != B.Value

Professor's Takeaway 🏠

Class, remember these **three important points**:

1. Comparison Based on Inequality

A Non-Equijoin does not use = as its main comparison operator.

It can use:

>

<

>=

<=

!=

2. Flexible Conditions

We can use different comparison conditions depending on the database problem.

The condition should match **what we are trying to determine**.

3. Useful for Complex or Range-Based Relationships

Non-Equi Joins are especially useful when one value needs to be matched against a **range**.

Examples:

Salary → Salary Grade

Score → Grade

Price → Discount

Age → Age Category

Income → Tax Bracket

Simplest definition to remember:

A Non-Equi Join is a SQL join that combines rows from different tables using comparison operators other than =, often to find relationships based on ranges or inequalities.

So, class, if you see words such as:

- salary range
- price range
- score range
- greater than
- less than
- within a range
- not equal

these are strong clues that the problem may involve a **Non-Equi Join**.

Important Clarification: != Does NOT Mean "No Matching Record"

There is one very important point that students often misunderstand.

Consider this condition:

WHERE E.EMP_ID != S.EMP_ID

This means:

Compare the employee ID with the salary table ID and keep the combinations where the IDs are different.

It does **not** mean:

"Find employees who do not have a salary record."

These are two different concepts.

Example

EMPLOYEE_TBL

EMP_ID NAME

1 Ana

2 Ben

SALARY_TBL

EMP_ID DATE_HIRE

1 2024-01-10

2 2024-02-15

Suppose we run:

SELECT

 E.EMP_ID,

 S.DATE_HIRE

FROM EMPLOYEE_TBL AS E

CROSS JOIN SALARY_TBL AS S

WHERE E.EMP_ID != S.EMP_ID;

The result can be:

E.EMP_ID S.DATE_HIRE

1 2024-02-15

2 2024-01-10

Why?

For Ana:

$1 \neq 1 \rightarrow \text{FALSE}$

$1 \neq 2 \rightarrow \text{TRUE}$

So Ana is paired with the salary-table row having ID 2.

For Ben:

$2 \neq 1 \rightarrow \text{TRUE}$

$2 \neq 2 \rightarrow \text{FALSE}$

So Ben is paired with the salary-table row having ID 1.

Notice:

Both employees **actually have matching EMP_IDs** in the salary table.

The query simply **excluded the equal combinations** because we used:

$\neq$

If the Goal Is "Find Employees Without a Matching Record"

If the actual question is:

"Which employees do not have a corresponding record in the Salary table?"

then $\neq$ is **not** the correct logic.

A more appropriate approach is:

SELECT

 E.EMP_ID

FROM EMPLOYEE_TBL AS E

LEFT JOIN SALARY_TBL AS S

 ON E.EMP_ID = S.EMP_ID

WHERE S.EMP_ID IS NULL;

Here, we are asking:

"Did this employee fail to find a matching record in the Salary table?"

Final Reminder 🏠

Do not confuse these two:

!=

E.EMP_ID != S.EMP_ID

Means:

"Are these two values different?"

LEFT JOIN + IS NULL

LEFT JOIN ...

WHERE S.EMP_ID IS NULL

Means:

"Does this employee have no matching record in the other table?"

They are not the same logic.

One-line memory trick:

!= means "different values."

LEFT JOIN + IS NULL means "no matching record."